

---

# Dossier de conception technique & architecture

*Jalon 4 – Projet Fil Rouge CDA*

---

**Étudiant** FORTUNATO Axel

**Formation** IPSSI — CDA Bachelor Fullstack DevOps

**Promotion** 2025–2026

**Date** Avril 2026

## Table des matières

---

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| 1. Objectif du jalon 4 . . . . .                                           | 2  |
| 2. Diagrammes UML – Cas d'utilisation (Use Cases) . . . . .                | 3  |
| 2.1 Acteurs . . . . .                                                      | 5  |
| 3. Diagrammes UML – Séquences (2 à 3 scénarios principaux) . . . . .       | 6  |
| 3.1 Séquence 1 – Lecture d'une page et sauvegarde de progression . . . . . | 6  |
| 3.2 Séquence 2 – Ajouter / retirer un favori . . . . .                     | 6  |
| 3.3 Séquence 3 – Lecture audio (TTS) d'une page . . . . .                  | 7  |
| 4. Diagramme de classes et couches applicatives (Back-end) . . . . .       | 8  |
| 4.1 Modèle métier (entités) . . . . .                                      | 8  |
| 4.2 Controllers, services, repositories (rôle) . . . . .                   | 8  |
| 5. Architecture multi-couches . . . . .                                    | 10 |
| 5.1 Pattern MVC (implémentation Symfony) . . . . .                         | 10 |
| 5.2 Architecture n-tiers (vue simple) . . . . .                            | 10 |
| 5.3 Séparation des responsabilités et bonnes pratiques . . . . .           | 11 |
| 5.4 Composants externes / bibliothèques . . . . .                          | 11 |
| 6. Schémas complémentaires (optionnel) . . . . .                           | 13 |
| 6.1 États de publication d'une histoire . . . . .                          | 13 |
| 7. Conclusion du jalon 4 . . . . .                                         | 13 |

## 1. Objectif du jalon 4

Ce document formalise la **conception technique** de l'application de lecture d'histoires pour enfants, en cohérence avec :

- le **CDCF** (Jalon 1) : périmètre fonctionnel et exigences,
- la **conception UI/UX** (Jalon 2) : écrans et parcours utilisateur,
- la **modélisation BDD** (Jalon 3) : MCD/MLD/MPD (entités User, Story, Page, Favorite, ReadingProgress).

Il fournit :

- les **diagrammes UML** (cas d'utilisation, séquences, entités métier),
  - la **description d'architecture multi-couches / n-tiers**,
  - les choix d'implémentation et l'intégration des composants externes (Text-to-Speech).
-

## **2. Diagrammes UML – Cas d'utilisation (Use Cases)**

Les cas d'utilisation ci-dessous couvrent l'ensemble des exigences fonctionnelles du CDCF (gestion utilisateurs, catalogue, lecture paginée, favoris/progression, administration, lecture audio).

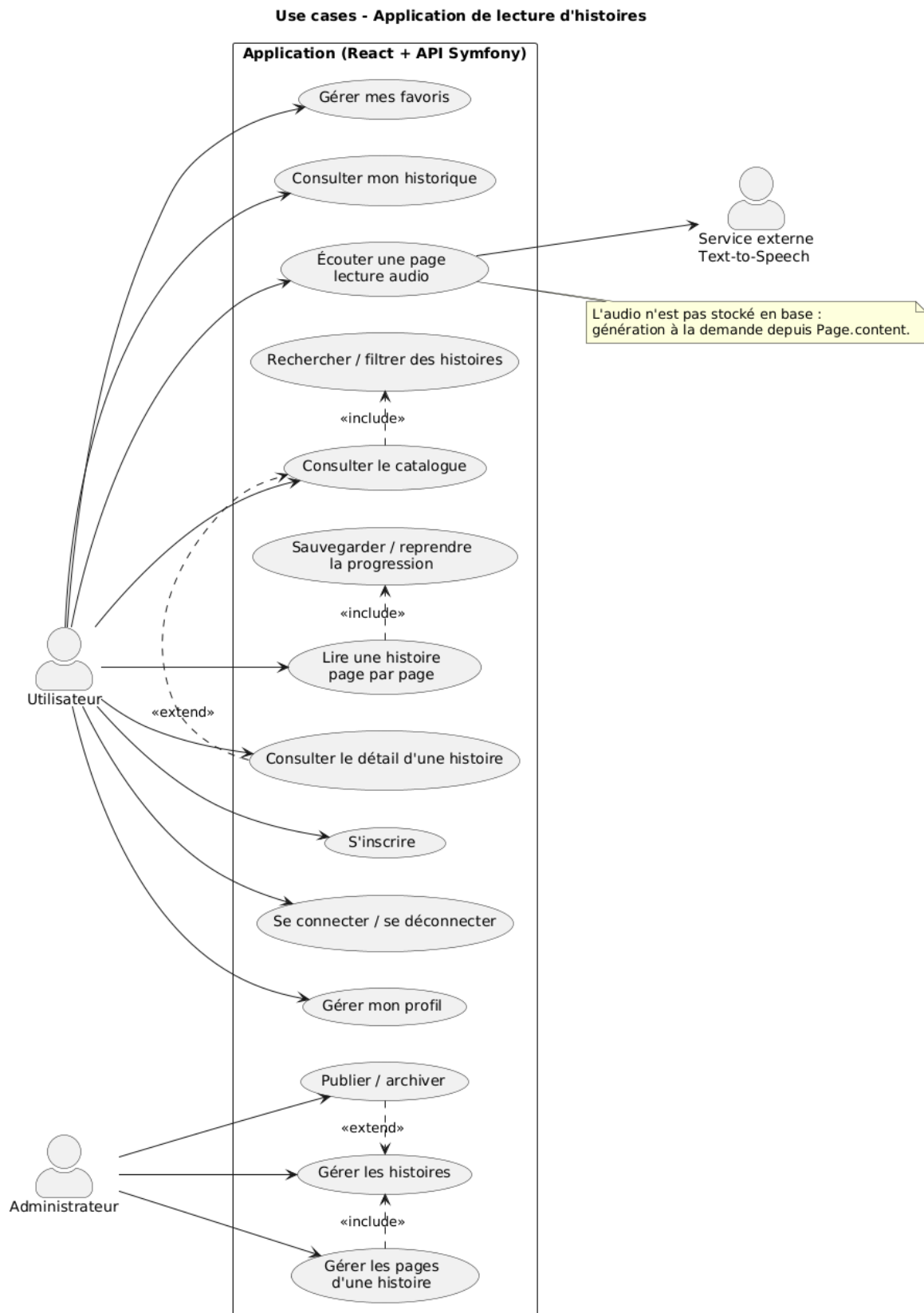


Fig. 1 : Diagramme UML – Cas d'utilisation

## 2.1 Acteurs

- **Utilisateur** : consulte le catalogue, lit les histoires, gère favoris et progression, utilise la lecture audio.
  - **Administrateur** : gère/modère les contenus (histoires et pages), publie/archivage.
  - **Service externe TTS** : API de synthèse vocale (acteur externe, appelé par le back-end).
-

### 3. Diagrammes UML – Séquences (2 à 3 scénarios principaux)

Les diagrammes suivants détaillent l'enchaînement des messages entre front-end React, API Symfony (controllers/services), Doctrine (repositories/entités) et la base MySQL.

#### 3.1 Séquence 1 – Lecture d'une page et sauvegarde de progression

Objectif : afficher une page, puis enregistrer la progression (reprise).

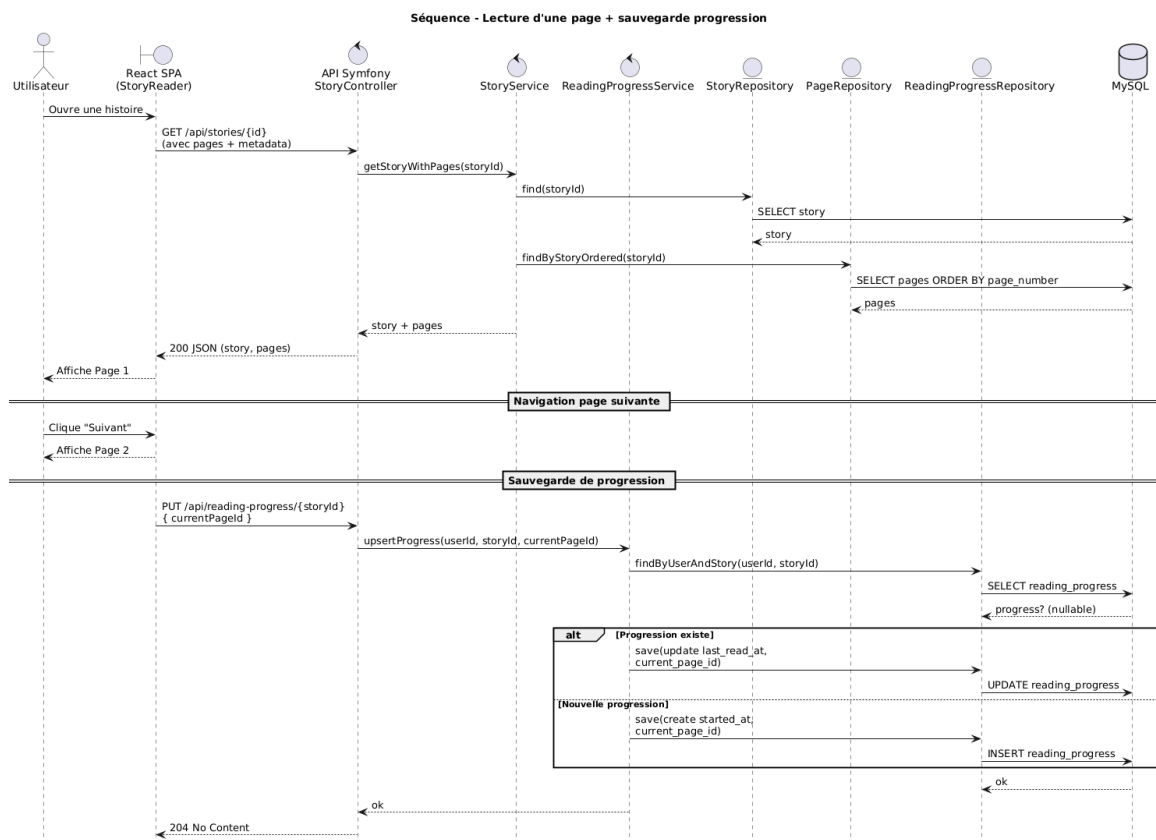


Fig. 2 : Diagramme UML – Séquence lecture + progression

#### 3.2 Séquence 2 – Ajouter / retirer un favori

Objectif : toggler un favori depuis le catalogue ou la page d'histoire.

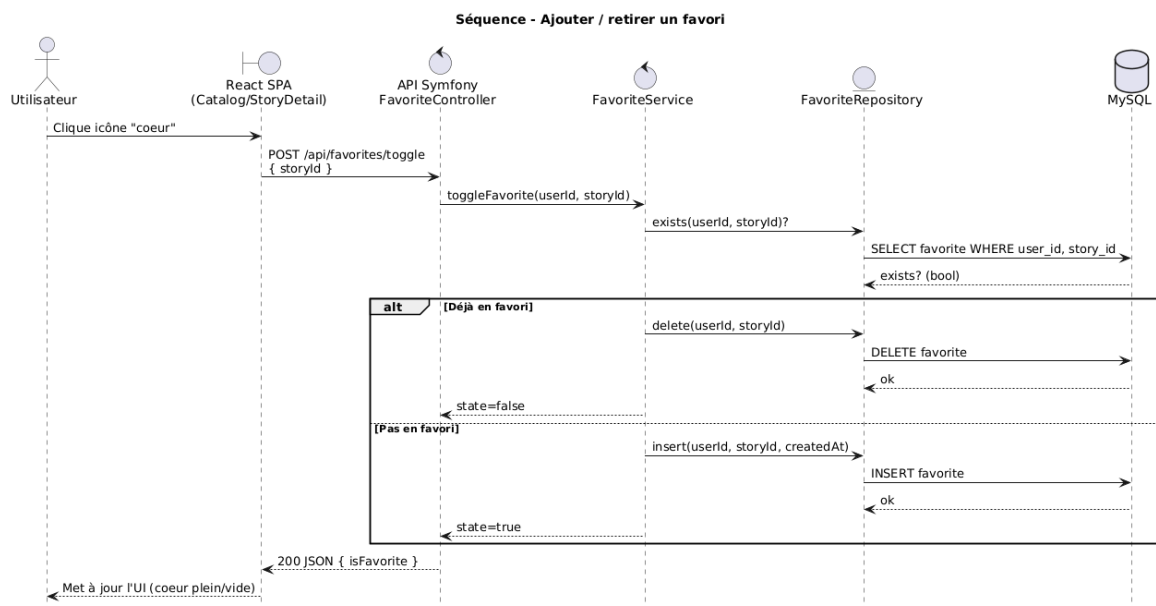


Fig. 3 : Diagramme UML – Séquence toggle favori

### 3.3 Séquence 3 – Lecture audio (TTS) d'une page

Objectif : demander la synthèse vocale pour une page (sans stocker d'audio en base).

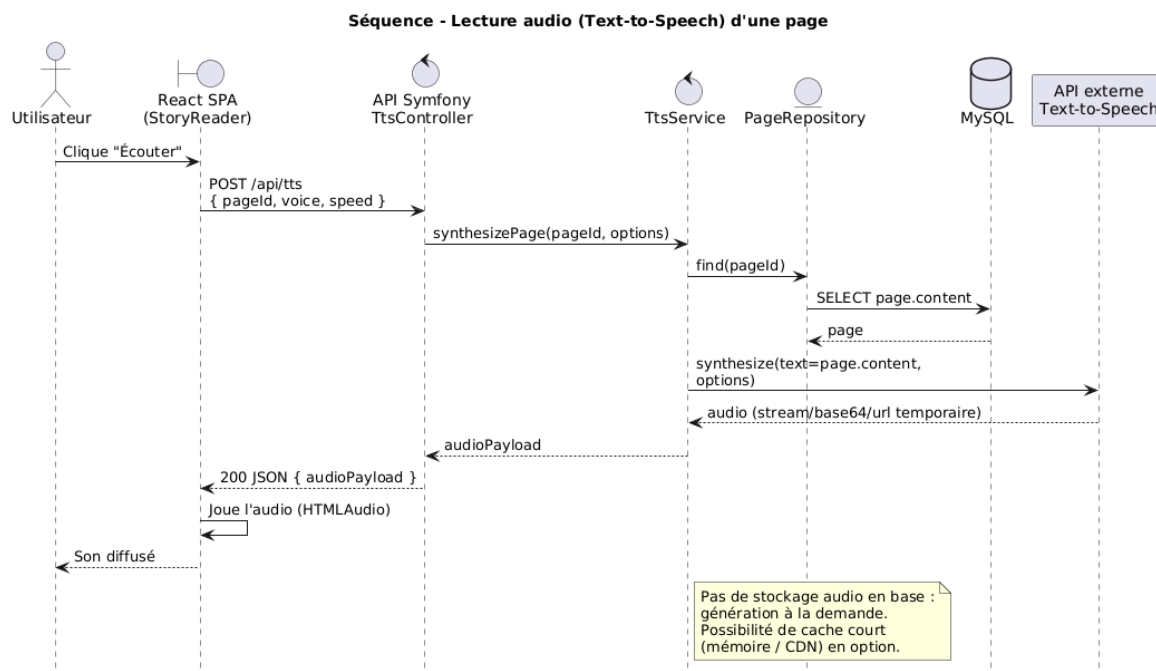


Fig. 4 : Diagramme UML – Séquence lecture audio (TTS)



## 4. Diagramme de classes et couches applicatives (Back-end)

### 4.1 Modèle métier (entités)

Le diagramme UML suivant représente **uniquement le domaine métier** : entités et associations telles que formalisées au Jalon 3 (MPD). Il ne détaille pas les classes techniques Symfony (controllers, services, repositories) afin de garder une lecture claire du **modèle de données objet**.

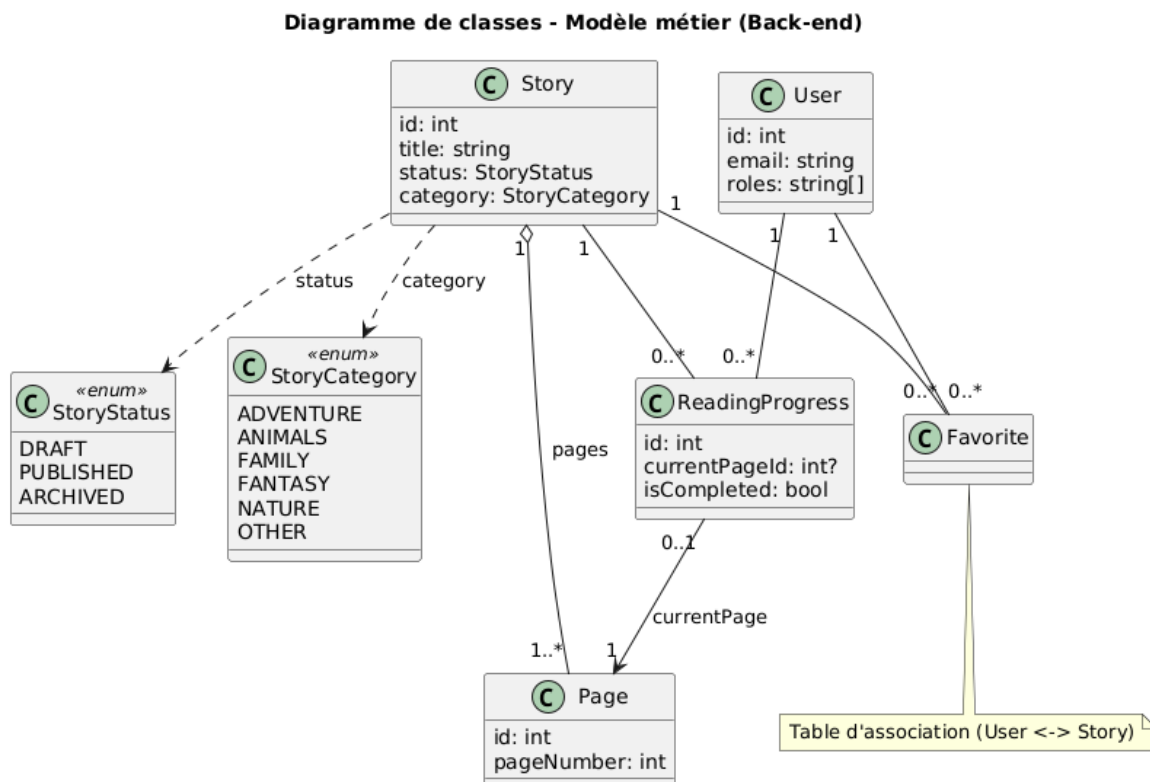


Fig. 5 : Diagramme UML – Entités métier

### 4.2 Controllers, services, repositories (rôle)

Les classes techniques Symfony ne figurent pas sur le diagramme d'entités §4.1 : elles organisent l'accès au domaine ainsi :

- **Controllers** (AuthController, StoryController, PageController, etc.) : points d'entrée HTTP, validation des entrées, réponses JSON.
- **Services métier** (ReadingProgressService, FavoriteService, StoryService...) : règles de progression, favoris, publication — orchestration sans SQL direct dans les contrôleurs.
- **Repositories Doctrine** : requêtes typées sur les entités ; transactions via l'ORM.

Enchaînement type : **SPA (fetch)** → **controller** → **service** → **repository** → **entités / MySQL**.

---

## 5. Architecture multi-couches

### 5.1 Pattern MVC (implémentation Symfony)

Même si l'application est une API REST (sans vues Twig), l'organisation Symfony reste alignée avec l'esprit MVC :

- **Controllers** : reçoivent les requêtes HTTP, valident les entrées (DTO/constraints), orchestrent l'appel aux services, renvoient une réponse JSON.
- **Services (métier)** : portent la logique applicative (règles de progression, règles favoris, publication, etc.).
- **Model / Accès données** : entités Doctrine + repositories + migrations.

Le **front-end React (SPA)** constitue la couche présentation (équivalent "View" dans une archi séparée).

### 5.2 Architecture n-tiers (vue simple)

Architecture logique 3-tiers (niveau "compréhensible par tous") :

- **Tier 1 – Client** : navigateur (React/TypeScript).
- **Tier 2 – Serveur applicatif** : API Symfony (PHP 8+) exposant une API REST.
- **Tier 3 – Données** : MySQL.

Important : **couches logiques** (controller/service/repository) ≠ **tiers physiques** (conteneurs). On peut déployer plusieurs couches logiques dans un même conteneur tout en conservant la séparation logique dans le code.

Schéma d'architecture (niveau "compréhensible par tous") :

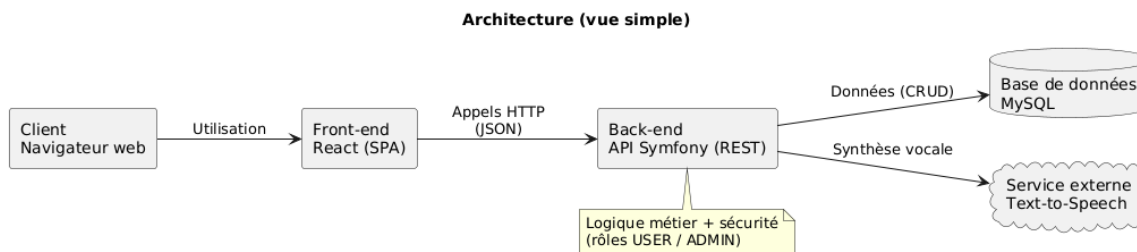


Fig. 6 : Schéma – Architecture 3-tiers

À l'intérieur du **tier 2** (API Symfony), une requête métier suit en général la pile : **Controller** → **Service métier** → **Repository Doctrine** → **base**. Ce flux vertical complète le schéma 3-tiers

ci-dessus (voir aussi §4.2).

### 5.3 Séparation des responsabilités et bonnes pratiques

Principes appliqués / prévus, avec **exemples concrets** :

| Principe                        | Illustration concrète                                                                                                                               |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SRP</b>                      | ReadingProgressService : règles de mise à jour de la progression; FavoriteService : ajout/retrait favori sans logique lecture dans le même service. |
| <b>DTO / validation</b>         | Corps JSON des routes POST/PATCH validés via classes Input + Symfony Validator avant passage au service (ex. pagination catalogue, toggle favori).  |
| <b>Secrets</b>                  | Clés DATABASE_URL, secrets JWT, clé API TTS uniquement dans .env / Docker ; jamais dans le dépôt.                                                   |
| <b>Doctrine / injection SQL</b> | Requêtes via QueryBuilder ou méthodes Repository avec paramètres liés.                                                                              |
| <b>Contrat API</b>              | Réponses JSON construites à partir des entités ou petits tableaux typés ; éviter d'exposer des champs internes (ex. hash mot de passe).             |
| <b>Tests</b>                    | PHPUnit sur services et webTestCase sur l'API ; Jest/RTL côté front (Jalon 5).                                                                      |

### 5.4 Composants externes / bibliothèques

Back-end (Symfony) :

- **Doctrine ORM** (entités/repositories/migrations),
- **Symfony Security** (authentification + rôles USER/ADMIN),
- **Authentification par JWT (Lexik JWT ou équivalent)** : retenu pour la **SPA React** consommant l'API en stateless (pas de session cookie côté API), en-têtes Authorization: Bearer, compatible déploiement et montée en charge. Les sessions serveur classiques restent plutôt adaptées au rendu HTML (Twig) ; ici l'API est JSON-only.

Externe :

- **Service de synthèse vocale (Text-to-Speech)** consommé par l'API.
- L'audio n'est **pas stocké en BDD** : génération à la demande à partir de `Page.content`.

Front-end :

- React + TypeScript,
  - appels HTTP (fetch/axios),
  - gestion d'état selon les besoins du front.
-

## 6. Schémas complémentaires (optionnel)

### 6.1 États de publication d'une histoire

Une histoire suit un cycle simple :

- DRAFT : visible uniquement en admin (préparation),
- PUBLISHED : visible dans le catalogue public,
- ARCHIVED : retirée du catalogue (conservée pour historique / audit).

Ce workflow est implémenté par `Story.status` (voir MPD Jalon 3).

---

## 7. Conclusion du jalon 4

À l'issue de ce jalon, la conception technique est suffisamment détaillée pour démarrer le développement :

- diagrammes UML (cas d'utilisation, séquences, entités métier) et description textuelle des couches Symfony (§4.2),
- architecture multi-couches claire (API REST + React SPA + MySQL, JWT),
- intégration prévue de l'API externe de synthèse vocale.